ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ C++

ΔΕΙΚΤΕΣ ΜΝΗΜΗΣ - Nodes

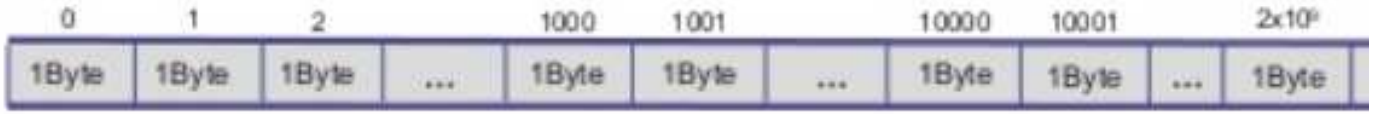
ΔΥΝΑΜΙΚΗ ΔΙΑΧΕΙΡΙΣΗ ΜΝΗΜΗΣ

ΜΕ ΔΕΙΚΤΕΣ ΜΝΗΜΗΣ

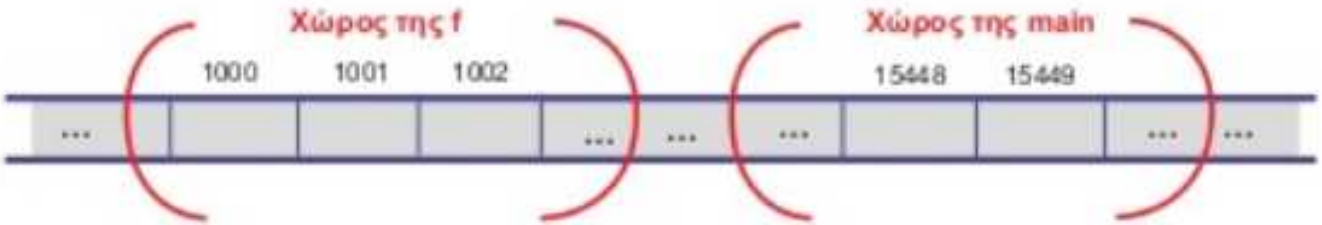
για την αναφορά σε πεδία

- Έχουμε ήδη δει κάποιες πληροφορίες για την οργάνωση της μνήμης σε ένα πρόγραμμα.

- Σε μια απλουστευμένη (αλλά επαρκή) προσέγγιση μπορούμε να φανταστούμε την μνήμη σαν μια ταινία με χώρους αποθήκευσης δεδομένων.
  - Κάθε χώρος αποθήκευσης είναι 1 byte (δηλαδή 8 bits).
  - Προκειμένου να μπορούμε να εντοπίσουμε κάθε χώρο αποθήκευσης, έχει έναν αναγνωριστικό αύξων αριθμό (που λέγεται διεύθυνση μνήμης).
  - Έτσι η εικόνα που πρέπει να έχουμε για την μνήμη θα είναι κελιά, το κάθε ένα με έναν αναγνωριστικό αριθμό, το οποίο θα έχει μέγεθος 1 byte:



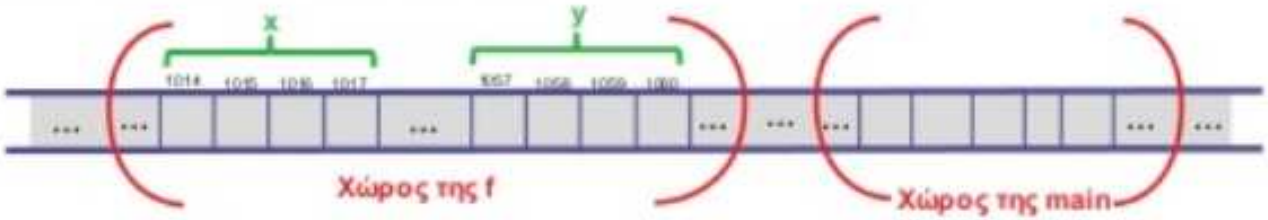
- Είδαμε ότι μια συνάρτηση έχει τον δικό της χώρο αποθήκευσης μεταβλητών.
  - Όταν πρόκειται να εκτελεστεί μια συνάρτηση, δεσμεύεται ο χώρος της μνήμης για τις μεταβλητές της συνάρτησης.
  - Έτσι αν για παράδειγμα η main καλεί μια συνάρτηση που έχουμε γράψει (έστω με όνομα f), τότε ο μεταγλωττιστής θα μεριμνήσει ώστε κάθε συνάρτηση να έχει τον χώρο της στην μνήμη.
  - Ο χώρος που δεσμεύεται είναι αρκετά μεγάλος για να χωρέσει πάρα πολλές μεταβλητές (το ακριβές πλήθος καθορίζεται από το λειτουργικό σύστημα, το μεταγλωττιστή κ.λπ.)



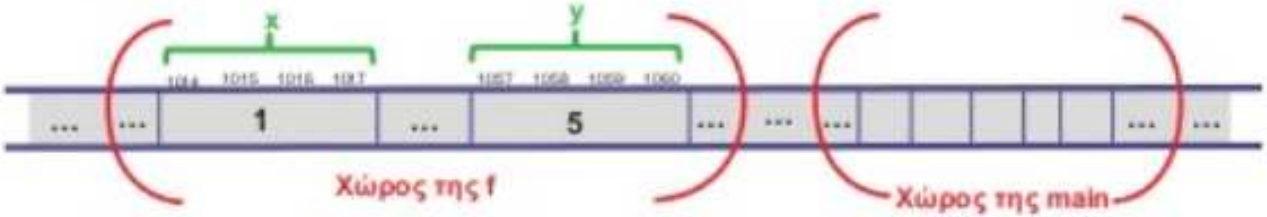
- Όταν δηλώνουμε μεταβλητές σε μια συνάρτηση, ο μεταγλωττιστής επιλέγει κάποιον από τον ελεύθερο χώρο της συνάρτησης για να αποθηκεύσει την μεταβλητή.

- Έτσι αν π.χ. η f δηλώνει στον χώρο δήλωσης μεταβλητών της δύο ακέραιες μεταβλητές, έστω την x και την y (θεωρούμε ότι μια ακέραια μεταβλητή δεσμεύει 4 bytes όπως συζητήσαμε σε προηγούμενο μάθημα)

- Τότε η εικόνα της μνήμης θα είναι η εξής

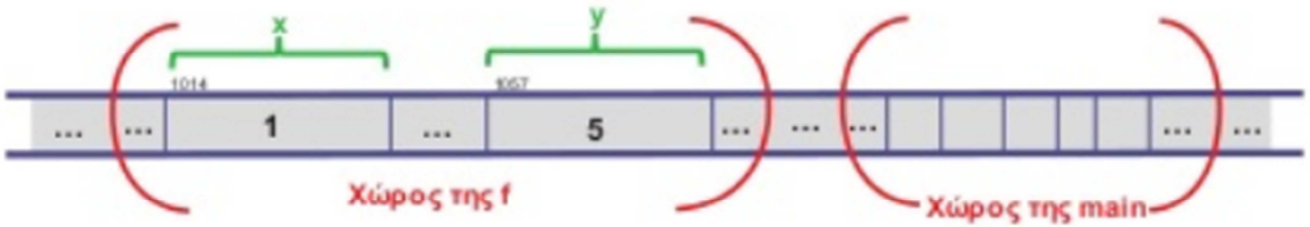


- Έτσι αν γράψουμε x=1 και y=5 με εντολές καταχώρησης στο σώμα της f, τότε αποθηκεύονται στις μεταβλητές οι αντίστοιχες τιμές.
- Ο τρόπος αποθήκευσης στο δυαδικό σύστημα των αριθμών αυτών δεν μας ενδιαφέρει όταν γράφουμε το πρόγραμμά μας.



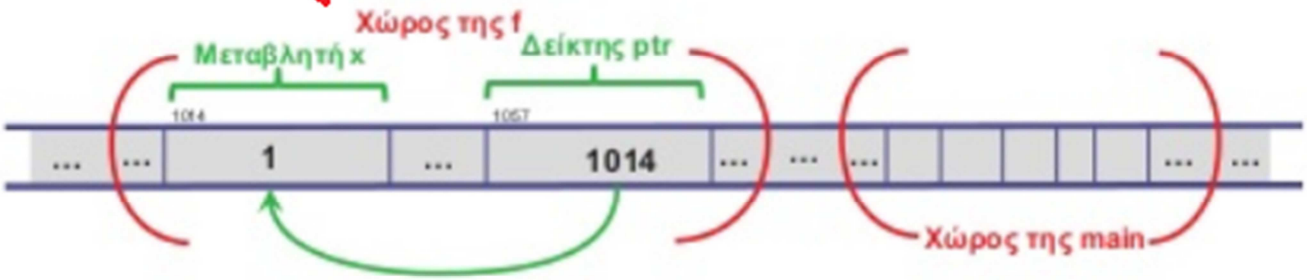
- Συνοψίζοντας αντιλαμβανόμαστε ότι μία μεταβλητή μπορεί να εντοπιστεί πλήρως από δύο δεδομένα: τον τύπο δεδομένων της (για να ξέρουμε πόσα bytes δεσμεύει στην μνήμη) και το πρώτο byte στο οποίο είναι αποθηκευμένη. Π.χ. Για την x αν ξέρω ότι είναι ακέραια και το 1ο byte της είναι το 1014, τότε μπορώ να εξάγω ότι θα είναι αποθηκευμένη στα bytes 1014-1017.

- Έτσι λοιπόν, η εικόνα που θα πρέπει να έχουμε στο μυαλό μας, είναι ότι κάθε μεταβλητή έχει έναν χώρο αποθήκευσης (ίσου με το μέγεθος της μεταβλητής σε bytes) με αναγνωριστικό τη διεύθυνση μνήμης του 1ου byte που αυτή δεσμεύει.



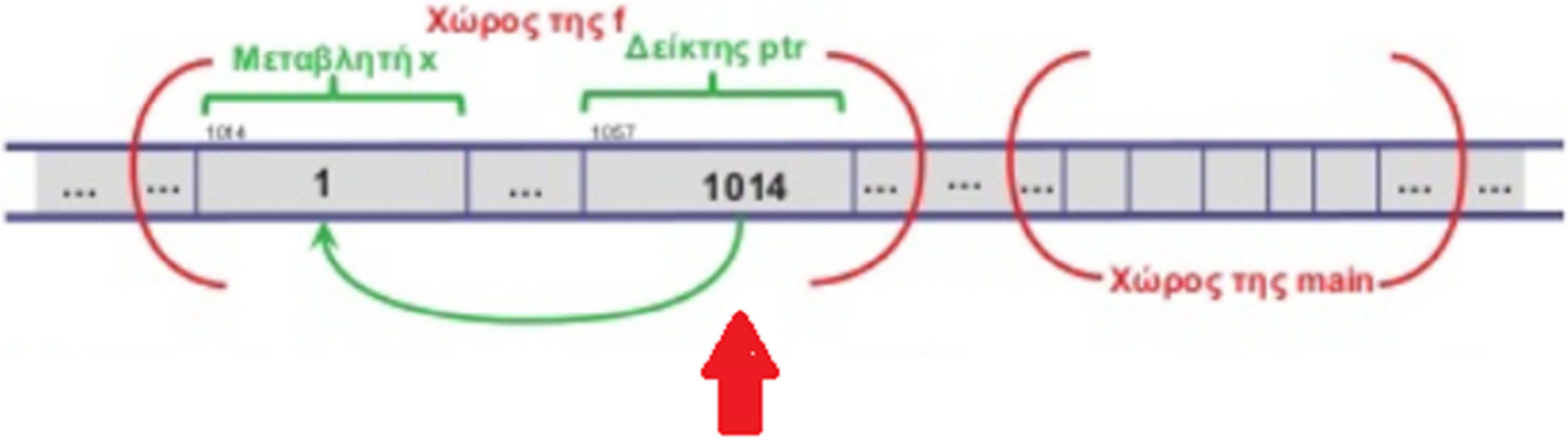
- Από την στιγμή όμως που η διεύθυνση μνήμης είναι ένας αριθμός, μπορεί να αντιμετωπιστεί σαν οποιοσδήποτε άλλος αριθμός στην C. Άρα για παράδειγμα μπορούμε να δηλώσουμε μια μεταβλητή στην οποία να αποθηκεύουμε την διεύθυνση μνήμης μιας άλλης μεταβλητής!

- Συνεπώς μπορούμε να δηλώσουμε μια μεταβλητή (έστω με όνομα ptr) που θα αποθηκεύσουμε π.χ. την διεύθυνση της μεταβλητής x.
- Αυτή η μεταβλητή, η οποία αποθηκεύει ως τιμή την διεύθυνση μνήμης άλλων μεταβλητών ονομάζεται δείκτης:



- Δείτε λοιπόν ότι ο δείκτης ptr έχει ως τιμή την διεύθυνση της μεταβλητής x. Θα λέμε για το λόγο αυτό, ότι ο δείκτης ptr δείχνει στην μεταβλητή x

Η ανωτέρω περιγραφή καταγράφει με ένα απλό παράδειγμα τον τρόπο με τον οποίο η γλώσσα προγραμματισμού C++ υλοποιεί την **<u>δυναμική διαχείριση μνήμης με δείκτες.</u>**



<u>δείκτης μνήμης</u>

μεταβλητή,

<u>η οποία αποθηκεύει ως τιμή την διεύθυνση μνήμης άλλων μεταβλητών</u>

Ειδικότερα,

- Ένας δείκτης δηλώνεται με την εντολή δήλωσης:

```
Τύπος_Δεδομένων *Ονομα_Δείκτη
```

  - Δηλώνουμε έναν δείκτη με όνομα «Ονομα_Δείκτη» που θα δείχνει σε μια μεταβλητή τύπου δεδομένων «Τύπος_Δεδομένων»

    Παραδείγματα:

    `int *p;` δηλώνουμε έναν δείκτη σε ακέραιο με όνομα p.

    `double *ptr;` δηλώνουμε έναν δείκτη σε double με όνομα ptr

- Διαπιστώστε ότι ένας δείκτης είναι ακόμη ένας τύπος δεδομένων. Με την ιδιαιτερότητα ότι οι μεταβλητές που θα αποθηκεύει θα είναι διευθύνσεις μνήμης.
- Ο τύπος δεδομένων συμβολικά είναι (int *) και (double *) αντίστοιχα στις δηλώσεις που είδαμε.
- Για να έχει νόημα ένας δείκτης πρέπει να τον συσχετίσουμε με μια μεταβλητή, ή όπως λέμε να τον βάλουμε να δείχνει στην μεταβλητή. Αυτό γίνεται με τον εξής τρόπο:

```
int x;
int *p;
p=&x; // Βάλε τον δείκτη p να δείχνει στην x.
```

  - Ο τελεστής & (θα τον διαβάζουμε «διεύθυνση του») μπαίνει μπροστά από μια μεταβλητή και μας επιστρέφει την διεύθυνση του.
  - Άρα:
    - αν η x είναι αποθηκευμένη π.χ. στην διεύθυνση 10333,
    - τότε η τιμή &x (διεύθυνση του x) είναι 10333,
    - άρα η τιμή της p μετά την εντολή ανάθεσης είναι 10333.

- Ο τελεστής * (που ονομάζεται τελεστής έμμεσης διευθυνσιοδότησης) έχει δύο χρήσεις:
  - την πρώτη που είναι να δηλώσουμε μια μεταβλητή τύπου δείκτη
  - δεύτερη χρήση είναι :

    Αν σε μία εντολή, βάλουμε το * μπροστά από έναν δείκτη, τότε αναφερόμαστε στην μεταβλητή που δείχνει ο δείκτης

- παράδειγμα:

```
int x;
int *p;
p=&x;  // Βάλε τον δείκτη p να δείχνει στην x
*p=5; //Η μεταβλητή που δείχνει το *p (άρα το x) παίρνει την τιμή 5
```

- Με αυτό το παράδειγμα αντιλαμβανόμαστε ότι έχουμε δύο τρόπους για πρόσβαση στην μεταβλητή x:
  - Απευθείας με το όνομα της x (άμεση πρόσβαση) στην μεταβλητή x
  - Μέσω του δείκτη που δείχνει στην x (έμμεση πρόσβαση) στην μεταβλητή x

```
/* pointers.c: Deixnei tin vasiki xrisi twn deiktwn */
#include <stdio.h>
main()
{  int x;
   int *ptr;

   ptr=&x;  // Βάλε τον δείκτη ptr να δείχνει στην x

   x=5;
      printf("x=%d\t   *ptr=%d\t &x=%d\t ptr=%d", x,*ptr, &x, ptr);
   *ptr=8;
      printf("\nx=%d\t *ptr=%d\t &x=%d\t ptr=%d", x,*ptr, &x, ptr);
   x=9;
      printf("\nx=%d\t *ptr=%d\t &x=%d\t ptr=%d", x,*ptr, &x, ptr);
}
```

ΤΑΥΤΟΧΡΟΝΗ ΧΡΗΣΗ ΑΝΑΦΟΡΑΣ ΜΕ ΔΕΙΚΤΗ ΜΝΗΜΗΣ

Στην C++ μπορούμε να κάνουμε ταυτόχρονη χρήση των αναφορών μαζί με τους δείκτες μνήμης (nodes).

Για παράδειγμα, εάν έχουμε την δήλωση,

```
int A{3};          // μη σταθερή αναφορά από ανώνυμη ποσότητα (αντί πεδίο, τιμή)
```

Το νέο πεδίο τύπου int με το όνομα **A**, έλαβε την ποσότητα 3, σαν μια, μη σταθερή αναφορά.

Μπορώ να αναφερθώ στο πεδίο **A**,

κάνοντας χρήση του ειδικού τελεστή &, που σημαίνει «εξαγωγή θέσης/διεύθυνσης μνήμης»

δηλαδή **&A**

και στην συνέχεια με ταυτόχρονη χρήση δείκτη

```
int A{3};
int * Z{&A};      //   1) το &A είναι η θέση μνήμης που βρίσκεται η A,
                  //   2) ακολουθεί η δήλωση ενός δείκτη, δηλαδή το int * Z
                  //   3) και στην συνέχεια γίνεται αναφορά με το int * Z{&A};
                  //      δηλαδή το A, έλαβε ένα δεύτερο όνομα, το Z, σαν αναφορά
                  //      αλλά γίνεται και μεταφορά (σαν ΑΝΑΘΕΣΗ τιμής) του address της A
                  //      με τα { } στον δείκτη Z
```